**Integrantes del Proyecto:**
Manuel Martín Rodrigo
Fernando Carrasco Carrasco
Manuel Fernando Quintero Posso
Manuel Castaño Rodríguez

**Fecha:** 14 de abril de 2026

# ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

*Universidad de Huelva*

**Métodos Formales en Ingeniería del Software**

Curso Académico 2025/2026

# PRÁCTICA 1

## Plataforma AutoHuelva: Memoria Técnica Detallada

# Índice

**1. Introducción y Objetivos** **2**

**2. Diseño Estructural: Diagrama de Clases** **3**
2.1. Tipos de Datos y Enumeraciones . . . . . 3
2.2. Clases Principales del Sistema . . . . . 3
2.2.1. Clase: vAutonomo . . . . . 3
2.2.2. Clase: Sensor . . . . . 4
2.2.3. Clase: Zona . . . . . 4
2.2.4. Clase: Pasajero . . . . . 4
2.2.5. Clase: Ruta . . . . . 5
2.3. Clases Adicionales (Mejora Creativa del Sistema) . . . . . 5
2.4. Asociaciones y Multiplicidades . . . . . 5

**3. Especificación Formal de Restricciones (Invariantes)** **6**
3.1. Restricciones Obligatorias (Reglas Base) . . . . . 6
3.2. Restricciones Creativas y Mejoras del Sistema . . . . . 7

**4. Formalización de Contratos de Operaciones** **9**

**5. Validación y Completitud (Configuraciones SOIL)** **10**

**6. Conclusiones y Trabajo Futuro** **16**

# 1. Introducción y Objetivos

La presente memoria técnica documenta de manera exhaustiva el desarrollo de la Práctica 1 de la asignatura Métodos Formales en Ingeniería del Software. El proyecto aborda la especificación formal de la plataforma AutoHuelva, un sistema diseñado para supervisar una flota de vehículos autónomos en un entorno urbano inteligente.

El objetivo primordial de esta práctica es adquirir habilidades avanzadas en el manejo del Lenguaje de Restricciones de Objetos (OCL). Mediante su aplicación sobre un diagrama de clases UML, se busca expresar la semántica del sistema para garantizar la seguridad de los pasajeros, el rigor de las normativas de tráfico y la eficiencia energética de la flota, evitando en todo momento que el sistema alcance estados incoherentes.

Para aportar una solución realista y original, el diseño no solo contempla el núcleo urbano de Huelva, sino que se ha conceptualizado para soportar trayectos con variaciones orográficas, como los desplazamientos hacia la sierra, exigiendo un control riguroso de la batería y el mantenimiento.

# 2. Diseño Estructural: Diagrama de Clases

El modelo estructural se ha desarrollado en la herramienta USE. A continuación, se detallan todas las enumeraciones, clases, atributos, métodos y asociaciones que componen la arquitectura del sistema.

## 2.1. Tipos de Datos y Enumeraciones

Para evitar ambigüedades y restringir los dominios de los atributos, se han definido las siguientes enumeraciones:

- **EstadoVehiculo**: {`Operativo, Mantenimiento`}. Define la disponibilidad física del vehículo.
- **ModoConduccion**: {`Autonomo, Manual`}. Determina quién tiene el control del vehículo.
- **TipoZona**: {`Peatonal, Carga, Emergencia, Estandar`}. Define las reglas de entorno aplicables.
- **TipoPasajero**: {`Estandar, Oro`}. Utilizado para la jerarquía de tarifas.
- **EstadoSensor**: {`Operativo, Fallo`}. Vital para la seguridad de la conducción autónoma.

## 2.2. Clases Principales del Sistema

El núcleo del sistema está compuesto por las siguientes entidades:

### 2.2.1. Clase: vAutonomo

Es la entidad central del sistema. Representa cada unidad de la flota.

- **Atributos:**
  - `idVehiculo : String` – Identificador único.
  - `velocidad : Real` – Velocidad actual en km/h.
  - `kilometrajeTot : Real` – Distancia total recorrida.
  - `estado : EstadoVehiculo` – Operativo o en mantenimiento.
  - `modo : ModoConduccion` – Autónomo o manual.
  - `nivelBateria : Real` – Porcentaje de carga (0-100).
  - `numAsientos : Integer` – Capacidad máxima de pasajeros.
  - `cierreSeguridad : Boolean` – Estado del bloqueo de puertas.
  - `maleteroDisponible : Boolean` – Capacidad de carga de equipaje.
  - `disponible : Boolean` – Indica si está libre para asignar rutas.
- **Operaciones:**
  - `arrancar() : Boolean` – Inicia la marcha si es seguro.

- `embarcarPasajero(p : Pasajero)` – Registra el ingreso de un usuario.
- `activarModoAutonomo() : Boolean` – Transfiere el control a la IA.
- `asignarRuta(r : Ruta)` – Vincula un trayecto al vehículo.
- `irACargar(e : EstacionCarga)` – Desplazamiento a punto de recarga.
- `establecerVelocidad(v : Real)` – Modifica la velocidad actual.
- `activarCierreSeguridad()` – Activa el bloqueo de puertas.
- `finalizarCarga() : Boolean` – Se invoca al alcanzar el 85 % de batería.

#### 2.2.2. Clase: Sensor

Representa el hardware de percepción del vehículo.

- **Atributos:**
  - `tipo : String`
  - `esCritico : Boolean`
  - `estado : EstadoSensor`

#### 2.2.3. Clase: Zona

Modela el entorno urbano y sus normativas geofísicas.

- **Atributos:**
  - `nombre : String`
  - `tipo : TipoZona`
  - `capacidadMax : Integer`
  - `velocidadMaxPermitida : Real`
- **Operaciones:**
  - `verificarCapacidad() : Boolean`.
  - `activarEmergencia()`.

#### 2.2.4. Clase: Pasajero

Representa a los clientes de la plataforma AutoHuelva.

- **Atributos:**
  - `idPasajero : String`
  - `edad : Integer`
  - `tipo : TipoPasajero`
  - `tieneEquipajePesado : Boolean`
  - `tarifa : Real`.
- **Operaciones:**
  - `calcularTarifa() : Real`
  - `registrarEquipaje(peso : Real)`
  - `solicitarTrayecto(r : Ruta)`

### 2.2.5. Clase: Ruta

Define el itinerario planificado.

- **Atributos:**
  - `idRuta : String`
  - `esPrioritaria : Boolean`
  - `tiempoEspera : Integer`.

## 2.3. Clases Adicionales (Mejora Creativa del Sistema)

Para cumplir con el requerimiento de aportar soluciones creativas y no triviales, se han incorporado cinco clases extra que dotan de gran realismo a la plataforma:

- **CentroControl:** Posee idCentro y estadoAlerta. Sus operaciones permiten emitirAlertaGlobal() y autorizarMantenimiento(), centralizando la seguridad del entorno inteligente.
- **EstacionCarga:** Cuenta con idEstacion y puntosLibres, además del método suministrarEnergia(). Modela la infraestructura física para cumplir las restricciones energéticas.
- **RegistroMantenimiento:** Contiene idRegistro y `descripcion`. Su método finalizarRevision() asegura que el ciclo de vida del vehículo tras los 5.000 km quede documentado.
- **InterfazPasajero:** Dotada de idioma y `avisoSeguridad`, permite mostrarAvisoSeguridad() directamente al usuario, vital para notificar el cierre por presencia de menores.
- **HistoricoTrayectos:** Almacena la distancia y el costeFinal, proveyendo la base de datos necesaria para generar resúmenes mediante generarResumen() y auditar la jerarquía de tarifas.

## 2.4. Asociaciones y Multiplicidades

Las entidades interactúan a través de relaciones estrictamente definidas:

- **VehiculoSensores** (`1` a `*`): Un vehículo está compuesto por múltiples sensores.
- **VehiculoPasajeros** (`0..1` a `*`): Un pasajero está en un único vehículo; el vehículo puede llevar varios pasajeros.
- **VehiculoZona** (`*` a `1`): Múltiples vehículos pueden ubicarse en la misma zona.
- **VehiculoRuta** (`1` a `0..1`): Un vehículo tiene asignada, como máximo, una ruta activa.
- **ZonaCentro** (`*` a `1`): Diversas zonas reportan al mismo Centro de Control.
- **VehiculoMantenimiento** (`1` a `*`): Historial de reparaciones del vehículo.
- **ZonaCarga** (`1` a `*`): Ubicación geográfica de las estaciones de suministro.
- **PasajeroInterfaz** (`1` a `1`): Cada usuario dispone de un terminal interactivo.
- **PasajeroHistorico** (`1` a `*`): Registro de viajes pasados del cliente.
- **VehiculoEstacion** (`*` a `0..1`): Vehículos vinculados temporalmente a un punto de carga.

# 3. Especificación Formal de Restricciones (Invariantes)

Esta sección documenta la formalización semántica de las reglas de negocio, divididas entre los requisitos obligatorios del enunciado y las ampliaciones creativas.

## 3.1. Restricciones Obligatorias (Reglas Base)

**1. Cierre de seguridad infantil:** Si en el grupo de pasajeros hay un niño con una edad inferior a 12 años, el vehículo debe activar obligatoriamente el cierre de seguridad.

```
context vAutonomo inv cierreInfantil:
    self.pasajeros->exists(p | p.edad < 12) implies self.cierreSeguridad
        = true
```

**2. Sensores en conducción autónoma:** Un vehículo en modo autónomo debe tener todos sus sensores críticos en estado operativo.

```
context vAutonomo inv sensoresCriticos:
    self.modo = ModoConduccion::Autonomo implies
    self.sensores->select(s | s.esCritico)->forAll(s | s.estado =
       EstadoSensor::Operativo)
```

**3. Mantenimiento por kilometraje:** Cuando un vehículo supera los 5.000 km totales pasa a mantenimiento y se retira de la circulación hasta que pase de nuevo al estado operativo.

```
context vAutonomo inv kmMantenimiento:
    self.kilometrajeTot > 5000 implies self.estado = EstadoVehiculo::
       Mantenimiento
```

**4. Control de aforo:** Un vehículo no puede arrancar si el número de pasajeros es mayor que el número de asientos disponibles.

```
context vAutonomo inv capacidadAsientos:
    self.velocidad > 0 implies self.pasajeros->size() <= self.
       numAsientos
```

**5. Capacidad máxima de zonas:** Ninguna zona puede albergar más vehículos que los que permite su capacidad máxima.

```
context Zona inv capacidadVehiculos:
    self.vehiculos->size() <= self.capacidadMax
```

**6. Tiempo de espera de rutas:** El tiempo total de espera no puede superar los 30 minutos.

```
context Ruta inv tiempoEsperaMax:
    self.tiempoEspera <= 30
```

**7. Logística de equipaje:** Si un pasajero lleva equipaje pesado, el vehículo debe tener el maletero disponible.

```
context vAutonomo inv maleteroEquipaje:
    self.pasajeros->exists(p | p.tieneEquipajePesado) implies self.
       maleteroDisponible = true
```

**8. Velocidad peatonal:** En las zonas de peatones, la velocidad máxima permitida no supera los 20 kms/hora.

```
context vAutonomo inv limitePeatones:
    self.zona.tipo = TipoZona::Peatonal implies self.velocidad <= 20
```

**9. Protocolo de batería crítica:** Si la batería del vehículo está por debajo del 10 %, el vehículo no debe aceptar pasajeros e ir a la zona más cerca de carga.

```
context vAutonomo inv bateriaBaja:
    self.nivelBateria < 10 implies (self.pasajeros->isEmpty() and self.
        zona.tipo = TipoZona::Carga)
```

**10. Jerarquía de tarifas:** La tarifa de un trayecto para un pasajero 'estandar' no puede ser inferior a la de un pasajero 'oro'.

```
context Pasajero inv tarifas:
    (self.tipo = TipoPasajero::Estandar and Pasajero.allInstances()->
        exists(p|p.tipo = TipoPasajero::Oro))
    implies self.tarifa >= Pasajero.allInstances()->select(p|p.tipo =
        TipoPasajero::Oro)->any(true).tarifa
```

**11. Protocolo de emergencias:** Cuando una zona cambia al estado de 'emergencia', el vehículo cambia a modo manual.

```
context vAutonomo inv zonaEmergencia:
    self.zona.tipo = TipoZona::Emergencia implies self.modo =
        ModoConduccion::Manual
```

**12. Restricción de rutas prioritarias:** Las rutas prioritarias solo pueden asignarse a vehículos con conducción autónoma.

```
context Ruta inv rutaPrioritaria:
    self.esPrioritaria implies self.vehiculo.modo = ModoConduccion::
        Autonomo
```

## 3.2. Restricciones Creativas y Mejoras del Sistema

Para asegurar la fiabilidad del modelo y añadir valor real, se han formalizado reglas de negocio avanzadas:

**13. Mantenimiento seguro:** Un vehículo en taller no puede contener pasajeros.

```
context vAutonomo inv mantenimientoSinPasajeros:
    self.estado = EstadoVehiculo::Mantenimiento implies self.pasajeros->
        isEmpty()
```

**14. Redundancia de alertas:** El Centro de Control debe reaccionar activando su alerta si una zona entra en emergencia.

```
context CentroControl inv alertaEmergencia:
    self.zonas->exists(z | z.tipo = TipoZona::Emergencia) implies self.
        estadoAlerta = true
```

**15. Físicas de recarga:** Un vehículo no puede estar en movimiento dentro de una estación de carga.

```
context vAutonomo inv cargaParado:
    self.zona.tipo = TipoZona::Carga implies self.velocidad = 0
```

**16. Límite de vida útil de batería (Mejora solicitada):** Para evitar la degradación de los acumuladores de litio, la carga se capa por software al 85 %.

```
context vAutonomo inv limiteCargaMax:
    self.nivelBateria <= 85
```

**17. Automatización de disponibilidad:** Al alcanzar el tope seguro del 85 %, el vehículo entra en servicio automáticamente.

```
context vAutonomo inv disponibleAlCargar:
    self.nivelBateria = 85 implies self.disponible = true
```

# 4. Formalización de Contratos de Operaciones

Para formalizar los contratos (pre-postcondiciones) de al menos 5 operaciones, se han modelado los métodos críticos de la clase `vAutonomo`:

**Operación: embarcarPasajero** Garantiza que no haya sobreaforo y que la batería sea suficiente antes de permitir el acceso.

```
context vAutonomo::embarcarPasajero(p : Pasajero)
    pre: self.pasajeros->size() < self.numAsientos
    pre: self.nivelBateria >= 10
    post: self.pasajeros->includes(p)
```

**Operación: activarModoAutonomo** Asegura mediante hardware el cambio de software de control.

```
context vAutonomo::activarModoAutonomo() : Boolean
    pre: self.sensores->select(s | s.esCritico)->forAll(s | s.estado =
       EstadoSensor::Operativo)
    post: (result = true implies self.modo = ModoConduccion::Autonomo)
```

**Operación: arrancar** Previene el movimiento si existen fallos logísticos de aforo.

```
context vAutonomo::arrancar() : Boolean
    pre: self.pasajeros->size() <= self.numAsientos
    pre: self.estado = EstadoVehiculo::Operativo
    post: self.velocidad > 0
```

**Operación: finalizarCarga** Controla la automatización del fin del suministro eléctrico.

```
context vAutonomo::finalizarCarga() : Boolean
    pre: self.zona.tipo = TipoZona::Carga
    pre: self.nivelBateria = 85
    post: self.disponible = true
    post: self.velocidad = 0
```

**Operación: asignarRuta** Controla la asignación de una ruta.

```
context vAutonomo::asignarRuta(r : Ruta)
    pre: self.disponible = true
    pre: self.estado = EstadoVehiculo::Operativo
    post: self.ruta = r
```

**Operación: activarEmergencia** Activa una emmergencia en una zona.

```
context Zona::activarEmergencia()
    pre: self.tipo <> TipoZona::Emergencia
    post: self.tipo = TipoZona::Emergencia
```

# 5. Validación y Completitud (Configuraciones SOIL)

Para construir configuraciones que permitan tener un nivel de certeza sobre la completitud del modelo lo más alto posible, se desarrollaron 4 scripts `.soil` que se evalúan en la consola de USE:

- **EscenarioValido.soil:** Genera una configuración estructuralmente sana. Crea un Centro de Control, una zona estándar, y un vehículo autónomo disponible con batería al 80 %, sensores LIDAR operativos y un pasajero adulto. *Resultado: Cero violaciones de invariantes.*
- **Fallo1.soil (Errores Críticos de Seguridad):** Instancia un niño sin activar `cierreSeguridad`, provoca un fallo de sensor en modo autónomo y establece la batería al 5 % con pasajeros dentro. *Resultado: El sistema intercepta las invariantes 1, 2 y 9.*
- **Fallo2.soil (Infracciones de Entorno):** Introduce un vehículo a 40 km/h en zona peatonal, asigna una ruta prioritaria en modo manual y vulnera la estructura económica haciendo que una tarifa Estandar sea inferior a la Oro. *Resultado: Se bloquean las reglas 8, 10 y 12.*
- **Fallo3.soil (Mantenimiento e Infraestructura):** Fuerza estados incoherentes entre las nuevas clases implementadas. Permite pasajeros en un coche en reparación, velocidad $> 0$ en zonas de carga, y oculta una alerta de emergencia al Centro de Control. *Resultado: El motor OCL valida positivamente el uso de las restricciones creativas al rechazar este estado.*

```
use> open EscenarioValido.soil
EscenarioValido.soil> -- 1. Crear infraestructura
EscenarioValido.soil> !new CentroControl('cc1')
EscenarioValido.soil> !set cc1.idCentro := 'CC-Huelva'
EscenarioValido.soil> !set cc1.estadoAlerta := false
EscenarioValido.soil>
EscenarioValido.soil> !new Zona('z1')
EscenarioValido.soil> !set z1.nombre := 'Centro'
EscenarioValido.soil> !set z1.tipo := TipoZona::Estandar
EscenarioValido.soil> !set z1.capacidadMax := 10
EscenarioValido.soil> !set z1.velocidadMaxPermitida := 50
EscenarioValido.soil>
EscenarioValido.soil> !insert (z1, cc1) into ZonaCentro
EscenarioValido.soil>
EscenarioValido.soil> -- 2. Crear veh├¡culo y sensor
EscenarioValido.soil> !new vAutonomo('v1')
EscenarioValido.soil> !set v1.idVehiculo := 'V-001'
EscenarioValido.soil> !set v1.velocidad := 40
EscenarioValido.soil> !set v1.kilometrajeTot := 1500
EscenarioValido.soil> !set v1.estado := EstadoVehiculo::Operat
EscenarioValido.soil> !set v1.modo := ModoConduccion::Autonomo
EscenarioValido.soil> !set v1.nivelBateria := 80
EscenarioValido.soil> !set v1.numAsientos := 4
EscenarioValido.soil> !set v1.cierreSeguridad := false
EscenarioValido.soil> !set v1.maleteroDisponible := true
EscenarioValido.soil> !set v1.disponible := true
EscenarioValido.soil>
EscenarioValido.soil> !new Sensor('s1')
EscenarioValido.soil> !set s1.tipo := 'LIDAR'
EscenarioValido.soil> !set s1.esCritico := true
EscenarioValido.soil> !set s1.estado := EstadoSensor::Operativ
EscenarioValido.soil>
EscenarioValido.soil> !insert (v1, z1) into VehiculoZona
EscenarioValido.soil> !insert (v1, s1) into VehiculoSensores
EscenarioValido.soil>
EscenarioValido.soil> -- 3. Crear pasajero y relaciones
EscenarioValido.soil> !new Pasajero('p1')
EscenarioValido.soil> !set p1.idPasajero := 'P-001'
EscenarioValido.soil> !set p1.edad := 25
EscenarioValido.soil> !set p1.tipo := TipoPasajero::Estandar
EscenarioValido.soil> !set p1.tieneEquipajePesado := false
EscenarioValido.soil> !set p1.tarifa := 15.0
EscenarioValido.soil>
EscenarioValido.soil> !new InterfazPasajero('ip1')
EscenarioValido.soil> !set ip1.idioma := 'ES'
EscenarioValido.soil> !set ip1.avisoSeguridad := false
EscenarioValido.soil>
EscenarioValido.soil> !insert (v1, p1) into VehiculoPasajeros
EscenarioValido.soil> !insert (p1, ip1) into PasajeroInterfaz
```

```
EscenarioValido.soil>
EscenarioValido.soil> -- Comprobar
EscenarioValido.soil> check
checking structure...
checked structure in 15ms.
checking invariants...
checking invariant (1) `CentroControl::alertaEmergencia': OK.
checking invariant (2) `Pasajero::tarifas': OK.
checking invariant (3) `Ruta::rutaPrioritaria': OK.
checking invariant (4) `Ruta::tiempoEsperaMax': OK.
checking invariant (5) `Zona::capacidadVehiculos': OK.
checking invariant (6) `vAutonomo::bateriaBaja': OK.
checking invariant (7) `vAutonomo::capacidadAsientos': OK.
checking invariant (8) `vAutonomo::cargaParado': OK.
checking invariant (9) `vAutonomo::cierreInfantil': OK.
checking invariant (10) `vAutonomo::kmMantenimiento': OK.
checking invariant (11) `vAutonomo::limitePeatones': OK.
checking invariant (12) `vAutonomo::maleteroEquipaje': OK.
checking invariant (13) `vAutonomo::mantenimientoSinPasajeros': OK.
checking invariant (14) `vAutonomo::sensoresCriticos': OK.
checking invariant (15) `vAutonomo::zonaEmergencia': OK.
checked 15 invariants in 0.047s, 0 failures.
EscenarioValido.soil>
```

*[EscenarioValido.soil]*

```
use> open Fallo1.soil
Fallo1.soil> !new Zona('z1')
Fallo1.soil> !set z1.tipo := TipoZona::Estandar
Fallo1.soil> !set z1.capacidadMax := 10
Fallo1.soil>
Fallo1.soil> !new vAutonomo('v1')
Fallo1.soil> !set v1.velocidad := 20
Fallo1.soil> !set v1.kilometrajeTot := 1000
Fallo1.soil> !set v1.estado := EstadoVehiculo::Operativo
Fallo1.soil> !set v1.modo := ModoConduccion::Autonomo
Fallo1.soil> !set v1.nivelBateria := 5  -- ERROR: Bater├¡a baja
Fallo1.soil> !set v1.numAsientos := 4
Fallo1.soil> !set v1.cierreSeguridad := false -- ERROR: No activado para ni├▒o
Fallo1.soil>
Fallo1.soil> !new Sensor('s1')
Fallo1.soil> !set s1.esCritico := true
Fallo1.soil> !set s1.estado := EstadoSensor::Fallo -- ERROR: Fallo en modo aut├│nomo
Fallo1.soil>
Fallo1.soil> !insert (v1, z1) into VehiculoZona
Fallo1.soil> !insert (v1, s1) into VehiculoSensores
Fallo1.soil>
Fallo1.soil> !new Pasajero('nino')
Fallo1.soil> !set nino.edad := 8 -- ERROR: Menor de 12
Fallo1.soil> !set nino.tipo := TipoPasajero::Estandar
Fallo1.soil> !set nino.tarifa := 10.0
Fallo1.soil>
Fallo1.soil> !insert (v1, nino) into VehiculoPasajeros
Fallo1.soil>
Fallo1.soil> check
checking structure...
Multiplicity constraint violation in association `PasajeroInterfaz':
  Object `nino' of class `Pasajero' is connected to 0 objects of class `InterfazPasajero'
  at association end `terminal' but the multiplicity is specified as `1'.
Multiplicity constraint violation in association `ZonaCentro':
  Object `z1' of class `Zona' is connected to 0 objects of class `CentroControl'
  at association end `centro' but the multiplicity is specified as `1'.
checked structure in 25ms.
```

```
checking invariants...
checking invariant (1) `CentroControl::alertaEmergencia': OK.
checking invariant (2) `Pasajero::tarifas': OK.
checking invariant (3) `Ruta::rutaPrioritaria': OK.
checking invariant (4) `Ruta::tiempoEsperaMax': OK.
checking invariant (5) `Zona::capacidadVehiculos': OK.
checking invariant (6) `vAutonomo::bateriaBaja': FAILED.
  -> false : Boolean
checking invariant (7) `vAutonomo::capacidadAsientos': OK.
checking invariant (8) `vAutonomo::cargaParado': OK.
checking invariant (9) `vAutonomo::cierreInfantil': FAILED.
  -> false : Boolean
checking invariant (10) `vAutonomo::kmMantenimiento': OK.
checking invariant (11) `vAutonomo::limitePeatones': OK.
checking invariant (12) `vAutonomo::maleteroEquipaje': OK.
checking invariant (13) `vAutonomo::mantenimientoSinPasajeros': OK.
checking invariant (14) `vAutonomo::sensoresCriticos': FAILED.
  -> false : Boolean
checking invariant (15) `vAutonomo::zonaEmergencia': OK.
checked 15 invariants in 0.049s, 3 failures.
Fallo1.soil>
```

*[Fallo1.soil]*

```
use> open Fallo2.soil
Fallo2.soil> !new Zona('zPeatonal')
Fallo2.soil> !set zPeatonal.tipo := TipoZona::Peatonal
Fallo2.soil> !set zPeatonal.capacidadMax := 5
Fallo2.soil>
Fallo2.soil> !new vAutonomo('v1')
Fallo2.soil> !set v1.velocidad := 40 -- ERROR: L├¡mite es 20 en peatonal
Fallo2.soil> !set v1.kilometrajeTot := 2000
Fallo2.soil> !set v1.estado := EstadoVehiculo::Operativo
Fallo2.soil> !set v1.modo := ModoConduccion::Manual
Fallo2.soil> !set v1.nivelBateria := 50
Fallo2.soil> !set v1.numAsientos := 4
Fallo2.soil>
Fallo2.soil> !insert (v1, zPeatonal) into VehiculoZona
Fallo2.soil>
Fallo2.soil> !new Ruta('r1')
Fallo2.soil> !set r1.esPrioritaria := true -- ERROR: Requiere modo aut├│nomo
Fallo2.soil> !set r1.tiempoEspera := 10
Fallo2.soil> !insert (v1, r1) into VehiculoRuta
Fallo2.soil>
Fallo2.soil> !new Pasajero('pOro')
Fallo2.soil> !set pOro.tipo := TipoPasajero::Oro
Fallo2.soil> !set pOro.tarifa := 50.0
Fallo2.soil>
Fallo2.soil> !new Pasajero('pEstandar')
Fallo2.soil> !set pEstandar.tipo := TipoPasajero::Estandar
Fallo2.soil> !set pEstandar.tarifa := 10.0 -- ERROR: Tarifa Estandar < Oro
Fallo2.soil>
Fallo2.soil> check
```

```
checking structure...
Multiplicity constraint violation in association `PasajeroInterfaz':
  Object `pOro' of class `Pasajero' is connected to 0 objects of class `InterfazPasajero'
  at association end `terminal' but the multiplicity is specified as `1'.
Multiplicity constraint violation in association `PasajeroInterfaz':
  Object `pEstandar' of class `Pasajero' is connected to 0 objects of class `InterfazPasajero'
  at association end `terminal' but the multiplicity is specified as `1'.
Multiplicity constraint violation in association `ZonaCentro':
  Object `zPeatonal' of class `Zona' is connected to 0 objects of class `CentroControl'
  at association end `centro' but the multiplicity is specified as `1'.
checked structure in 37ms.
checking invariants...
checking invariant (1) `CentroControl::alertaEmergencia': OK.
checking invariant (2) `Pasajero::tarifas': FAILED.
  -> false : Boolean
checking invariant (3) `Ruta::rutaPrioritaria': FAILED.
  -> false : Boolean
checking invariant (4) `Ruta::tiempoEsperaMax': OK.
checking invariant (5) `Zona::capacidadVehiculos': OK.
checking invariant (6) `vAutonomo::bateriaBaja': OK.
checking invariant (7) `vAutonomo::capacidadAsientos': OK.
checking invariant (8) `vAutonomo::cargaParado': OK.
checking invariant (9) `vAutonomo::cierreInfantil': OK.
checking invariant (10) `vAutonomo::kmMantenimiento': OK.
checking invariant (11) `vAutonomo::limitePeatones': FAILED.
  -> false : Boolean
checking invariant (12) `vAutonomo::maleteroEquipaje': OK.
checking invariant (13) `vAutonomo::mantenimientoSinPasajeros': OK.
checking invariant (14) `vAutonomo::sensoresCriticos': OK.
checking invariant (15) `vAutonomo::zonaEmergencia': OK.
checked 15 invariants in 0.049s, 3 failures.
Fallo2.soil>
```

*[Fallo2.soil]*

```
use> open Fallo3.soil
Fallo3.soil> !new CentroControl('cc1')
Fallo3.soil> !set cc1.estadoAlerta := false -- ERROR: Deber├¡a ser true por la emergencia
Fallo3.soil>
Fallo3.soil> !new Zona('zEmergencia')
Fallo3.soil> !set zEmergencia.tipo := TipoZona::Emergencia
Fallo3.soil> !set zEmergencia.capacidadMax := 10
Fallo3.soil> !insert (zEmergencia, cc1) into ZonaCentro
Fallo3.soil>
Fallo3.soil> !new Zona('zCarga')
Fallo3.soil> !set zCarga.tipo := TipoZona::Carga
Fallo3.soil> !set zCarga.capacidadMax := 5
Fallo3.soil> !insert (zCarga, cc1) into ZonaCentro
Fallo3.soil>
Fallo3.soil> !new vAutonomo('vEnMantenimiento')
Fallo3.soil> !set vEnMantenimiento.kilometrajeTot := 6000
Fallo3.soil> !set vEnMantenimiento.estado := EstadoVehiculo::Mantenimiento
Fallo3.soil> !set vEnMantenimiento.velocidad := 0
Fallo3.soil> !set vEnMantenimiento.numAsientos := 4
Fallo3.soil> !insert (vEnMantenimiento, zEmergencia) into VehiculoZona
Fallo3.soil>
Fallo3.soil> !new Pasajero('p1')
Fallo3.soil> !set p1.edad := 30
Fallo3.soil> !insert (vEnMantenimiento, p1) into VehiculoPasajeros -- ERROR: Pasajero en coche en mantenimiento
Fallo3.soil>
Fallo3.soil> !new vAutonomo('vEnCarga')
Fallo3.soil> !set vEnCarga.estado := EstadoVehiculo::Operativo
Fallo3.soil> !set vEnCarga.velocidad := 15 -- ERROR: Deber├¡a ser 0 en zona de carga
Fallo3.soil> !set vEnCarga.kilometrajeTot := 100
Fallo3.soil> !set vEnCarga.numAsientos := 4
Fallo3.soil> !insert (vEnCarga, zCarga) into VehiculoZona
Fallo3.soil>
Fallo3.soil> check
```

```
checking structure...
Multiplicity constraint violation in association `PasajeroInterfaz':
  Object `p1' of class `Pasajero' is connected to 0 objects of class `InterfazPasajero'
  at association end `terminal' but the multiplicity is specified as `1'.
checked structure in 14ms.
checking invariants...
checking invariant (1) `CentroControl::alertaEmergencia': FAILED.
  -> false : Boolean
checking invariant (2) `Pasajero::tarifas': OK.
checking invariant (3) `Ruta::rutaPrioritaria': OK.
checking invariant (4) `Ruta::tiempoEsperaMax': OK.
checking invariant (5) `Zona::capacidadVehiculos': OK.
checking invariant (6) `vAutonomo::bateriaBaja': FAILED.
  -> false : Boolean
checking invariant (7) `vAutonomo::capacidadAsientos': OK.
checking invariant (8) `vAutonomo::cargaParado': FAILED.
  -> false : Boolean
checking invariant (9) `vAutonomo::cierreInfantil': OK.
checking invariant (10) `vAutonomo::kmMantenimiento': OK.
checking invariant (11) `vAutonomo::limitePeatones': OK.
checking invariant (12) `vAutonomo::maleteroEquipaje': OK.
checking invariant (13) `vAutonomo::mantenimientoSinPasajeros': FAILED.
  -> false : Boolean
checking invariant (14) `vAutonomo::sensoresCriticos': OK.
checking invariant (15) `vAutonomo::zonaEmergencia': FAILED.
  -> false : Boolean
checked 15 invariants in 0.052s, 5 failures.
Fallo3.soil>
```

*[Fallo3.soil]*

# 6. Conclusiones y Trabajo Futuro

La realización de esta práctica ha permitido consolidar los conocimientos teóricos sobre Métodos Formales y el estándar OCL. La transición de un diagrama de clases estático a un modelo dinámico y restringido demuestra la potencia de la validación formal en las fases tempranas de la Ingeniería del Software.

Como reflexiones sobre la solución o problemas encontrados, el uso de enumeraciones ha resultado vital para acotar los dominios de los atributos, evitando la ambigüedad que supondría el uso de cadenas de texto libres. Asimismo, la separación de responsabilidades mediante la creación de clases adicionales (como *CentroControl* o *RegistroMantenimiento*) ha facilitado la redacción de restricciones mucho más realistas.

**Posibles ampliaciones:** Como trabajo futuro y posibles ampliaciones de la propuesta, se plantea refinar la gestión energética de la flota. Sería conveniente implementar restricciones que limiten la carga máxima de las baterías al 85 % para prolongar su vida útil, añadiendo un cambio de estado automático a 'disponible' una vez alcanzado este umbral, integrándose de forma natural con los atributos ya previstos en la clase *vAutonomo*.